Command Line Arguments

Instead of invoking the input statement from inside the program, it is possible to pass data from the command line to the main() function when the program is executed. These values are called **command line arguments**.

Command line arguments are important for your program, especially when you want to control your program from outside, instead of hard coding those values inside the code.

Let us suppose you want to write a C program "hello.c" that prints a "hello" message for a user.

Instead of reading the name from inside the program with scanf(), we wish to pass the name from the command line as follows −

```
C:\users\user>hello Prakash
```

The string will be used as an argument to the main() function and then the "Hello Prakash" message should be displayed.

argc and argv

To facilitate the main() function to accept arguments from the command line, you should define two arguments in the main() function – **argc** and **argv[]**.

argc refers to the number of arguments passed and **argv[]** is a pointer array that points to each argument passed to the program.

```
int main(int argc, char *argv[]) {
    ...
    ...


    return 0;
}
```

The **argc** argument should always be non-negative. The **argv** argument is an array of character pointers to all the arguments, argv[0] being the name of the program. After that till "**argv [argc - 1]**", every element is a command-line argument.

```c
#include <stdio.h>

int main(int argc, char* argv[])
{
    printf("Program name is: %s" , argv[0]);

    if (argc == 1)
        printf("\nNo Extra Command Line Argument Passed "
               "Other Than Program Name" );

    if (argc >= 2) {
        printf("\nNumber Of Arguments Passed: %d" , argc);
        printf("\n----Following Are The Command Line "
               "Arguments Passed----" );
        for (int i = 0; i < argc; i++)
            printf("\nargv[%d]: %s" , i, argv[i]);
    }
    return 0;
}
```

Terminal Input `$ ./a.out First Second Third`

Output

```
Program Name Is: ./a.out
Number Of Arguments Passed: 4
----Following Are The Command Line Arguments Passed----
argv[0]: ./a.out
argv[1]: First
argv[2]: Second
argv[3]: Third
```